Análise de Otimização em Processamento Alto Desempenho

Parte I – Análise Conceitual

Questão 1 – Otimização não significa apenas «fazer menos contas»

Supondo que A e B rodam no mesmo computador, o programa B pode ter uma melhor localidade espacial que o A, como é o exemplo de casos de acesso de matrizes por colunas (pior localidade) e por linhas (melhor localidade). Nesse caso, talvez os acessos realizados por A, mesmo que menores, tenham uma localidade pior.

O programa A talvez esteja carregando na cache dados mas apenas usando um desses dados em cada operação, necessitando então carregar novos dados para então realizar as demais operações.

A estrutura de dados de A pode ser menos otimizada que a de B, como nos casos em que fazer busca de acessos em uma lista gera um custo maior do que fazer em um set.

O programa A pode estar gerando um overhead ao chamar funções que o programa B não chama.

Questão 2 – Hierarquia de memória

a) Por que o acesso à RAM é mais demorado que o acesso aos caches L, que por sua vez são mais demorados que os acessos aos registradores: A hierarquia de memória funciona baseando-se no princípio de localidadade. Quanto menor e mais rápido o nível de memória, maior o custo em termos de capacidade e latência.

b) Um cache miss ocorre quando o dado buscado não está no nível acessado, forçando a CPU a buscar o dado em um nível abaixo. Exemplo: buscar em L3 um dado que está na RAM.

c) Um cache hit ocorre quando o dado buscado está no nível consultado.

d) Localidade Espacial: Os vizinhos dos dados acessados têm a chance de serem acessados. Localidade Temporal: Os dados agora acessados têm a chance de serem acessados novamente em breve.

e) O primeiro padrão de acesso faz uso sequencial, o que gera uma localidade espacial melhor, pois os dados estão carregados em sequência e também uma localidade temporal melhor, pois como o acesso está ocorrendo sequencialmente, o próximo dado será acessado em seguida. O segundo padrão tem um stride muito grande e por isso a memória cache precisa ser constantemente atualizada, levando a muito mais cache miss.

Parte II – Otimização Sequencial

Questão 3 – Análise do código

Ao comparar as três versões de implementação:

```
# Versão A - Lista dinâmica com math.sqrt(x[i]) + math.sqrt(x[i])
resultado = []
for i in range(len(x)):
    valor = math.sqrt(x[i]) + math.sqrt(x[i])
    resultado.append(valor)   # Alocação dinâmica a cada append

# Versão B - Pré-alocação + acesso direto
resultado = [0.0] * len(x)
for i in range(len(x)):
    valor = 2 * math.sqrt(x[i])
    resultado[i] = valor      # Acesso direto

# Versão C - NumPy com pré-alocação otimizada
size = len(x)
resultado = np.empty(size, dtype=np.float64)
for i in range(size):
    valor = 2 * math.sqrt(x[i])
    resultado[i] = valor
```

A principal diferença está na alocação de memória: em vez de declarar uma lista vazia e fazer a alocação prévia do espaço de memória, deve-se pré-alocar com tamanho conhecido. O comando `append()` realoca toda a memória anterior mais o espaço para o novo elemento a cada execução. Como o tamanho e tipo de dado são conhecidos, fazer a pré-alocação é melhor. Além disso, `2*math.sqrt(x[i])` reduz o overhead comparado a duas chamadas separadas.

Resultados Experimentais

Foram executados benchmarks com tamanhos crescentes ($N = 10^4$, 10^5, 10^6, 10^7) de entrada, com cinco execuções replicadas para cada ponto. A análise considera a velocidade relativa do tempo médio entre as implementações.

Os resultados demonstram que a Versão C (NumPy) tem o melhor desempenho consistente em todos os tamanhos de entrada, alcançando acelerações significativas sobre a Versão A:

- Para $N = 10^5$: aceleração de 9.25x em relação à Versão A
- Para $N = 10^6$: aceleração de 9.73x em relação à Versão A
- Acima de $N = 10^7$: desaceleração observada devido a efeitos de memória hierárquica

A Versão B pré-alocada apresenta melhoria consistente sobre a Versão A, mas sem o desempenho da implementação NumPy. A Versão C demonstra que estruturas de dados otimizadas e pré-alocação proporcionam acelerações consistentes ao eliminar realocações constantes de memória.

Parte III – Cache e Organização da Memória

Questão 6 – Previsão antes do experimento

O algoritmo 2 (percurso por colunas) tem melhor desempenho localidade espacial, onde os elementos são carregados sequencialmente nas caches. As linhas são carregadas contigamente na memória, então o vizinho do elemento atual provavelmente já está carregado na memória. Já na implementação A (por linhas), há um risco maior de cache miss devido ao stride irregular ao acessar por colunas.

Questão 7 – Experimento

Dado $T_{colunas} = R \times T_{linhas}$, o valor R representa para um dado N quanto que $T_{columnas}$ muda em relação proporcionalmente a T_{linhas}.

- Se $0 < R < 1$, então $T_{colunas}$ é necessariamente menor que T_{linhas}
- Se $R = 1$, então $T_{colunas} = T_{linhas}$
- Se $R > 1$, então $T_{colunas} > T_{linhas}$

No experimento, R é sempre menor que 1, indicando que a execução por colunas é mais rápida que a execução por linhas. Isso contradintivamente confirma que a localidade espacial no acesso por colunas proporciona melhor desempenho, uma vez que elementos de memória vizinhos são acessados em sequência, maximizando cache hits.

Questão 12 – Comportamento Quadrático

O crescimento quadrático indica uma equação polinomial de segundo grau do tipo:

$$y = ax^2 + bx + c$$

Apesar dos polinômios terem o mesmo grau, a maneira como eles «abrem» varia drasticamente com o valor a. Por essa razão, um algoritmo de ordem 2 com $0 < a < 1$ tem um crescimento temporal muito menor que algoritmos de ordem 2 com $a > 1$. Isso considera que operações mais eficientes em memória e melhor localidade espacial resultam em menor fator quadrático no tempo de execução.

Análise de Código - Benchmarks

Foram executados os seguintes códigos:

```
# Versão A
@njit
def calcula_A(x):
    resultado = []
    for i in range(len(x)):
        valor = math.sqrt(x[i]) + math.sqrt(x[i])
```

```
        resultado.append(valor)
    return resultado

# Versão B
@njit
def calcula_B(x):
    resultado = [0.0] * len(x)
    size = len(x)
    for i in range(size):
        valor = 2 * math.sqrt(x[i])
        resultado[i] = valor
    return resultado

# Versão C
@njit
def calcula_C(x):
    size = len(x)
    resultado = np.empty(size, dtype=np.float64)
    for i in range(size):
        valor = 2 * math.sqrt(x[i])
        resultado[i] = valor
    return resultado

# Benchmark execution com tamanhos de entrada:
# (10_000, 100_000, 1_000_000, 10_000_000)
```

Os resultados demonstram que a Versão C (implementada em NumPy) alcança acelerações significativas:

N	mean_A	mean_B	mean_C
10,000	0.062795s	0.015114s	0.022909s
100,000	0.002750s	0.002767s	0.000297s
1,000,000	0.027806s	0.026668s	0.002858s
10,000,000	0.251521s	0.241323s	0.030476s

Questão 5 – Speedup

Análise das acelerações obtidas:

a) A versão C (implementada em NumPy) apresenta melhor desempenho.

b) Somente para a versão B é comparável à versão A sem melhoria significativa. c) Para C com base no aumento do número de dados: espera-se que exista uma demora maior de processamento quando os dados aumentam na ordem de 10x, o que explica o declínio visto entre 10^6 e 10^7. O speedup apresenta um salto entre 10^4 e 10^5. d) A versão NumPy utiliza uma estrutura de dados diferente, sobrescrevendo a maneira como operadores padrões funcionam (ex: []). Assim, apesar do código ser semelhante, sua execução é logicamente diferente.

Parte III – Cache e Organização da Memória

```
# Algoritmo 1 - percurso por linhas
@njit
def sum_A(A, N):
    soma = 0.0
    for i in range(N):
        for j in range(N):
            soma += A[i, j]

# Algoritmo 2 - percurso por colunas
@njit
def sum_B(A, N):
    soma = 0.0
    for j in range(N):
        for i in range(N):
            soma += A[i, j]
```

Q6 - Previsão antes do experimento: O algoritmo 2 por conta da localidade espacial apresenta melhor desempenho, carregando elementos sequencialmente nas caches.

Questão 12 – Experimento

R é uma constante de proporção entre $T_{colunas}$ e T_{linhas}, ou seja:

$$T_{colunas} = R \times T_{linhas}$$

No experimento com matrizes de ordem N = (500, 1000, 2000, 3000, 4000):

N	A	B	speedupA	speedupB
500	0.000002s	0.000001s	1.0	3.35
1000	0.000001s	0.000001s	1.0	2.50
2000	0.000002s	0.000001s	1.0	4.47
3000	0.000002s	0.000001s	1.0	3.42
4000	0.000003s	0.000001s	1.0	6.12

Note que speedupB > 1 indica que $T_{colunas} > T_{linhas}$. Isso confirma que a execução por colunas é mais rápida, o que pode inicialmente parecer contraditório, mas reflete melhor padrões de acesso com menor overhead.

Questão 7 – Análise de Curvas Quadráticas

O crescimento O(N^2) indica uma equação polinomial de segundo grau:

$$y = ax^2 + bx + c$$

As curvas demonstram que, mesmo com o mesmo grau, a maneira como polinômios «abrem» varia drasticamente com o valor a. Um algoritmo de ordem 2 com $0 < a < 1$ apresenta crescimento temporal menor que algoritmos com $a > 1$.